1. Popis aplikace, motivace

Popis aplikace

Navrhovaná aplikace je jednoduchý internetový obchod (e-shop), který umožňuje uživatelům prohlížet produkty, vyhledávat zboží, vytvářet objednávky a sledovat historii nákupů. Systém je rozdělen na samostatné části (mikroservisy), které zajišťují jednotlivé funkce aplikace.

Motivace

Motivací projektu je navrhnout a implementovat e-shop, na kterém lze demonstrovat principy moderního návrhu softwarových systémů. Projekt se zaměřuje na rozdělení aplikace do samostatných částí, jejich vzájemnou komunikaci a práci s daty. Cílem je získat praktické zkušenosti s návrhem architektury a vývojem aplikace.

---

2. Funkční požadavky

FRQ1 – Registrace uživatele
Systém umožňuje registraci nového uživatele zadáním potřebných údajů.

FRQ2 – Přihlášení uživatele
Systém umožňuje přihlášení registrovaného uživatele pomocí emailu a hesla.

FRQ3 – Prohlížení produktů
Systém poskytuje seznam všech dostupných produktů.

FRQ4 – Vyhledávání produktů
Systém umožňuje vyhledávat produkty podle názvu nebo kategorie.

FRQ5 – Zobrazení detailu produktu
Systém umožňuje zobrazit detailní informace vybraného produktu.

FRQ6 – Vytvoření objednávky
Systém umožňuje uživateli vytvořit novou objednávku z produktů přidaných do košíku.

FRQ7 – Zobrazení historie objednávek
Systém umožňuje uživateli zobrazit historii svých předchozích objednávek.

FRQ8 – Aktualizace dostupnosti produktu
Systém automaticky aktualizuje dostupnost produktu po vytvoření objednávky.

FRQ9 – Správa produktů administrátorem
Administrátor může přidávat nové produkty, upravovat existující produkty nebo odstraňovat produkty ze systému.

---

3. Nefunkční požadavky

NFR1 – Škálovatelnost
Architektura systému musí umožňovat horizontální škálování jednotlivých mikroservis podle potřeby, například při zvýšeném počtu uživatelů nebo požadavků na systém.

NFR2 – Bezpečnost
Přístup k systému musí být zabezpečen autentizací uživatelů (např. Basic Authentication nebo JWT), aby k funkcím systému měli přístup pouze oprávnění uživatelé.

NFR3 – Modularita
Jednotlivé mikroservisy by měly fungovat nezávisle na sobě. Pokud dojde k výpadku nebo odstranění jedné služby, ostatní části systému zůstanou funkční v rámci své dostupné funkcionality.

NFR4 – Udržovatelnost
Systém musí být navržen tak, aby bylo možné snadno provádět změny a údržbu bez výrazného zásahu do ostatních částí systému.

---

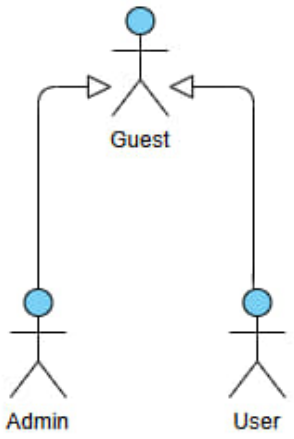
4. Seznam uživatelů

V systému se vyskytují tři typy uživatelů, kteří mají různé role a oprávnění.

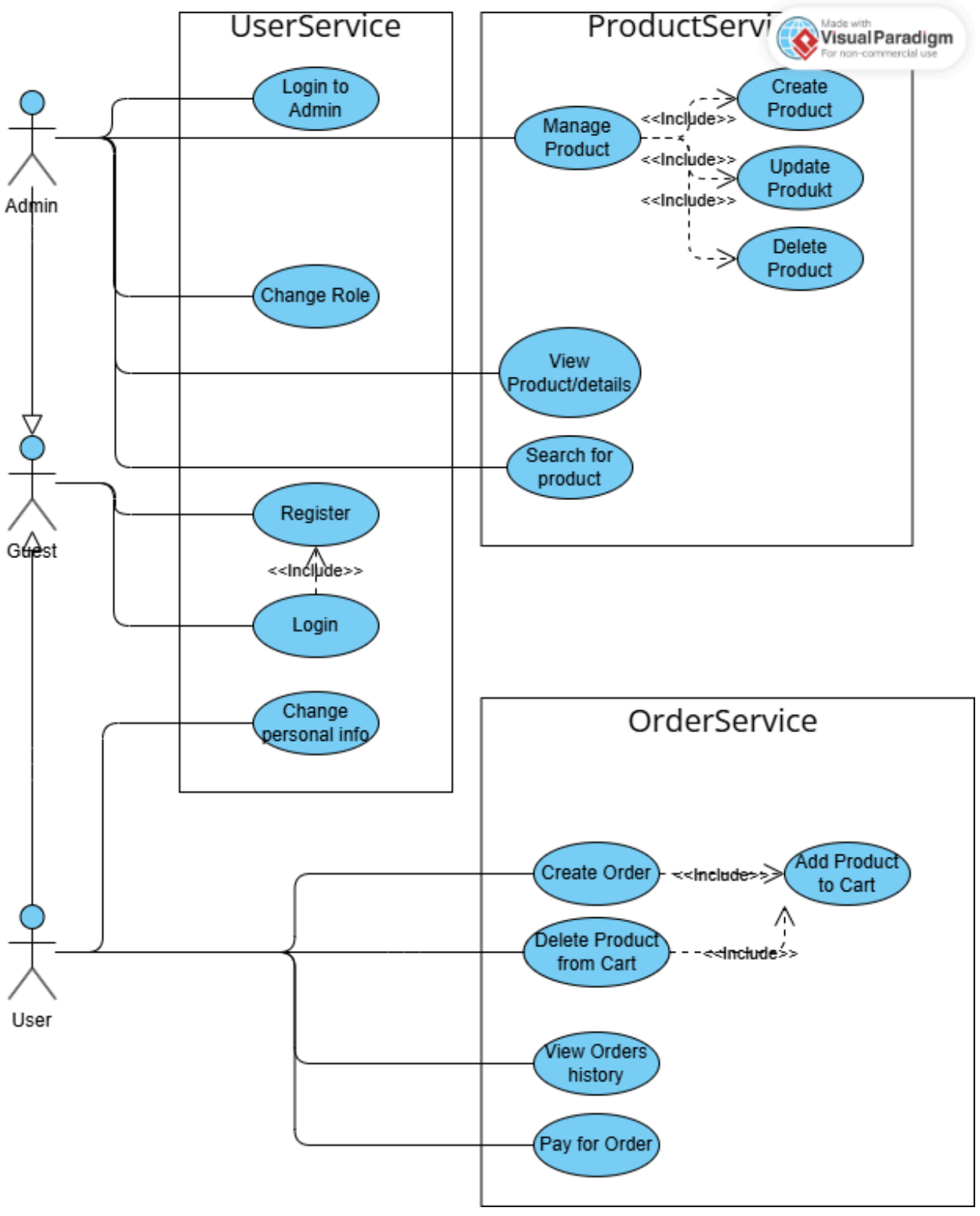
1. Návštěvník (Guest) - je nepřihlášený uživatel systému. Může prohlížet nabídku produktů a vyhledávat zboží, ale nemůže vytvářet objednávky ani provádět další pokročilé operace.

2. Zákazník (User) - je registrovaný uživatel systému, který využívá e-shop k nákupu produktů. Může prohlížet nabídku produktů, vyhledávat produkty, přidávat je do košíku a vytvářet objednávky. Má také možnost zobrazit historii svých objednávek.

3. Administrátor (Admin) - je uživatel s rozšířenými oprávněními, který je zodpovědný za správu obsahu e-shopu. Může přidávat nové produkty, upravovat existující produkty a odstraňovat produkty ze systému. Jeho úkolem je udržovat systém aktuální a správně spravovaný.



5. Případy užití (Use Cases)



UC1 – Registrace uživatele

Primární aktér: Zákazník

Popis:
Uživatel si může vytvořit nový účet v systému zadáním registračních údajů.

Základní scénář:

1. Uživatel otevře registrační formulář.
2. Uživatel zadá požadované údaje (např. jméno, email, heslo).
3. Systém ověří správnost zadaných údajů.
4. Systém vytvoří nový uživatelský účet v databázi.
5. Systém informuje uživatele o úspěšné registraci.

UC2 – Přihlášení uživatele

Primární aktér: Zákazník

Popis:
Registrovaný uživatel se může přihlásit do systému pomocí svého emailu a hesla.

Základní scénář:

1. Uživatel otevře přihlašovací formulář.
2. Uživatel zadá email a heslo.
3. Systém ověří přihlašovací údaje.
4. Systém přihlásí uživatele do systému.
5. Uživatel získá přístup k funkcím aplikace.

UC3 – Vytvoření objednávky

Primární aktér: Zákazník

Popis:
Uživatel může vytvořit objednávku vybraných produktů.

Základní scénář:

1. Uživatel vybere produkt.
2. Uživatel přidá produkt do košíku.
3. Uživatel přejde do košíku a potvrdí objednávku.
4. Systém vytvoří novou objednávku v databázi.
5. Systém zobrazí potvrzení o úspěšném vytvoření objednávky.

UC4 – Zobrazení historie objednávek

Primární aktér: Zákazník

Popis:
Uživatel může zobrazit seznam svých předchozích objednávek.

Základní scénář:

1. Uživatel otevře sekci historie objednávek.
2. Systém načte seznam objednávek z databáze.
3. Systém zobrazí seznam objednávek uživateli.

UC5 – Přidání produktu do košíku

Primární aktér: Zákazník

Popis:
Uživatel může přidat vybraný produkt do svého nákupního košíku.

Základní scénář:

1. Uživatel zobrazí detail produktu.
2. Uživatel klikne na tlačítko „Přidat do košíku“.
3. Systém přidá produkt do košíku uživatele.
4. Systém zobrazí aktualizovaný obsah košíku.

UC6 – Správa produktu

Primární aktér: Administrátor

Popis:
Administrátor může spravovat produkty v systému (přidávat, upravovat nebo odstraňovat produkty).

Základní scénář:

1. Administrátor se přihlásí do administrátorského rozhraní.
2. Administrátor otevře sekci správy produktů.
3. Administrátor může přidat nový produkt, upravit existující produkt nebo odstranit produkt.
4. Systém uloží provedené změny do databáze.

---

6. Výběr vhodné architektury (architektonický styl)

Pro tento projekt byla zvolena **architektura mikroservis (Microservice Architecture)**. Tento architektonický styl rozděluje aplikaci na několik menších samostatných služeb, které jsou zodpovědné za konkrétní část funkcionality systému.

V navrhovaném systému budou jednotlivé části aplikace rozděleny do následujících mikroservis:

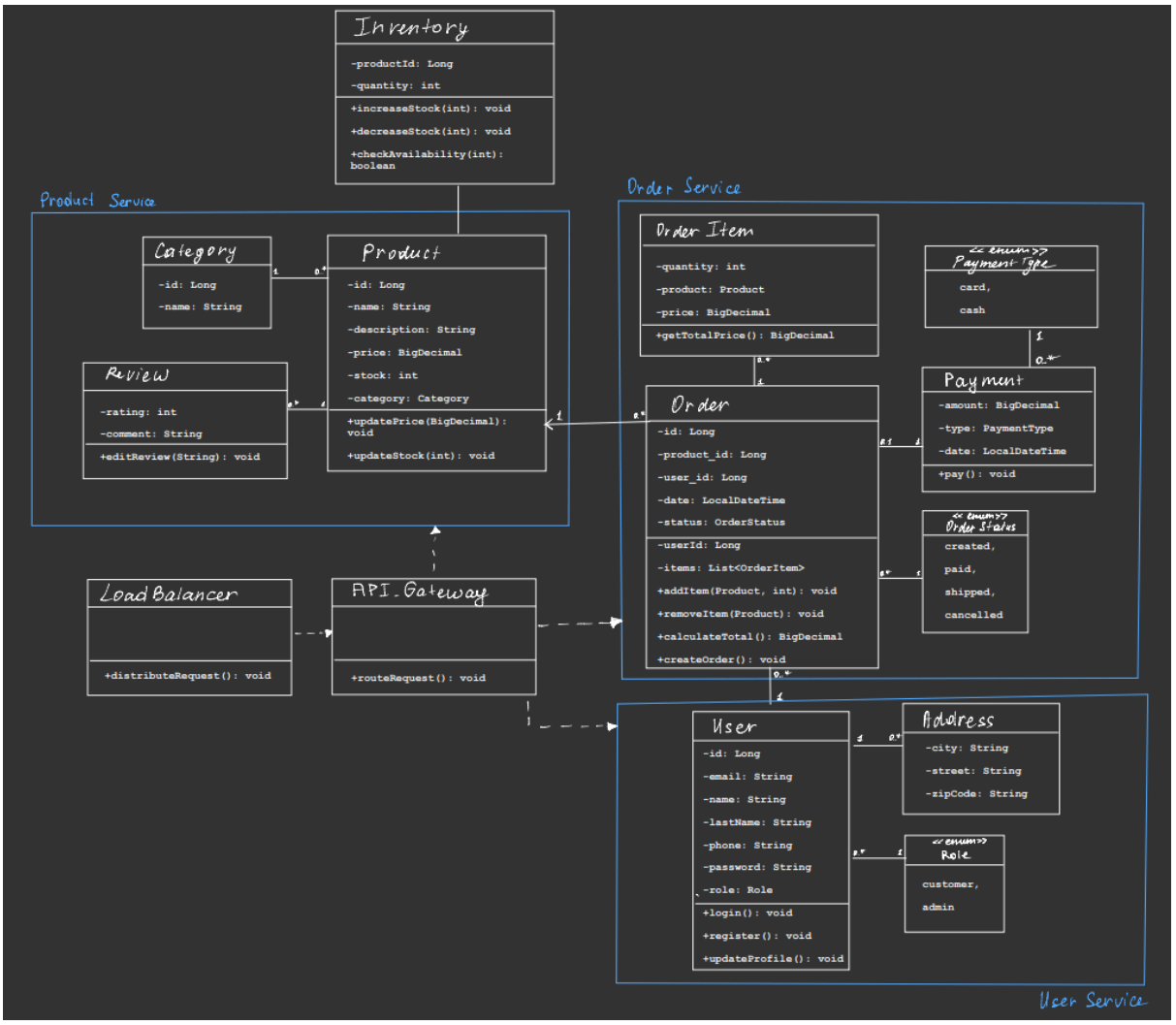
- **User Service** – správa uživatelů
- **Product Service** – správa produktů
- **Order Service** – správa objednávek

Každá mikroservisa má vlastní databázi, což zajišťuje nezávislost jednotlivých služeb.

Komunikace mezi jednotlivými službami bude probíhat prostřednictvím **REST API** a v některých případech také pomocí **messaging systému** (např. Apache Kafka nebo RabbitMQ), který umožní předávání informací mezi službami asynchronně.

Architektura je doplněna o API Gateway a Load Balancer, které zajišťují směrování požadavků a rovnoměrné rozložení zátěže.

7. UML class diagram



8. Component diagram

